Object relationships

Each room belongs to a venue, so to create a `Room` object using the API (or in the docs interface), you need to associate it with a `Venue`. To do this, use the numeric ID of the related object. A `RoomIn` schema would include a field named `venue`, which you populate with the integer ID of the room's related `Venue` object.

11.2.7 Updating and deleting with the API

Two letters down, two to go; it's time to address *U* and *D*. Updating an object is rather similar to creating a new one—in fact, if you're using the PUT method, the same serializer can be used. As you're editing an existing object, include the ID of the object to be changed in the URL. Ninja doesn't provide any shortcuts for updating the fields on an object, so you need to loop through the payload and apply each field using a call to `setattr()`. The following listing contains a new API endpoint that updates all the field values on an existing `Venue` instance.

NOTE `setattr()` is a Python built-in method that allows you to set a property on an object by giving its name as a string and the new value.

The following listing contains a new API endpoint that updates all the field values on an existing `Venue` instance.

Listing 11.14 A update endpoint for venues

```
# RiffMates/bands/api.py
...

@router.put("/venue/{venue_id}/",
    response=VenueOut, auth=api_key)
def update_venue(request, venue_id, payload:VenueIn):
    venue = get_object_or_404(Venue, id=venue_id)
    for key, value in payload.dict().items():
        setattr(venue, key, value)

    venue.save()
    return venue
```

Include the ID of the venue to be edited as a URL argument.

The response of an update call is typically the altered object.

Like with POST, specify an inbound payload.

Fetch the object to be updated.

Loop through the key–value pairs in the payload, and apply each to the object.

Remember to save your changes to the object before returning it.

Ninja supports both the PUT and PATCH methods as decorators. You seldom need to implement both. PUT gets used when the client provides all fields, whereas PATCH only sends fields that changed. PATCH takes up less bandwidth but typically requires its own schema. A `ModelSchema` subclass decides which fields are required based on its ORM Model. For example, if *first_name* is a required field, it will be required in the schema. To implement a PATCH, required fields need to be optional instead, as you are only sending the changed values. There is no easy way to do this, so you have to